

Accès aux documents .csv et prévisualisation

In [634... `import pandas as pd`

In [635... `#Création des variables permettant l'accès aux documents CSV`
`population = pd.read_csv('population.csv')`

In [636... `aide_alimentaire = pd.read_csv('aide_alimentaire.csv')`
`dispo_alimentaire = pd.read_csv('dispo_alimentaire.csv')`
`sous_nutrition = pd.read_csv('sous_nutrition.csv')`

In [637... `#Taille du document`
`print('le tableau contient {} observation(s) ou article(s)'.format(population.shape`
`print('le tableau contient {} colonne(s)'.format(population.shape[1]))`

le tableau contient 1416 observation(s) ou article(s)
le tableau contient 3 colonne(s)

In [638... `#Affichage des 5 premières lignes`
`display(population.head())`

	Zone	Année	Valeur
0	Afghanistan	2013	32269.589
1	Afghanistan	2014	33370.794
2	Afghanistan	2015	34413.603
3	Afghanistan	2016	35383.032
4	Afghanistan	2017	36296.113

In [639... `#Conversion de la valeur population en unités, non en milliers`
`population['Valeur'] = population['Valeur']*1000`

In [640... `display(population.head())`

	Zone	Année	Valeur
0	Afghanistan	2013	32269589.0
1	Afghanistan	2014	33370794.0
2	Afghanistan	2015	34413603.0
3	Afghanistan	2016	35383032.0
4	Afghanistan	2017	36296113.0

```
In [641... #Renommage de la colonne en un nom plus explicite  
population.rename(columns={'Valeur': 'Population'})
```

```
Out[641...  


|      | Zone        | Année | Population |
|------|-------------|-------|------------|
| 0    | Afghanistan | 2013  | 32269589.0 |
| 1    | Afghanistan | 2014  | 33370794.0 |
| 2    | Afghanistan | 2015  | 34413603.0 |
| 3    | Afghanistan | 2016  | 35383032.0 |
| 4    | Afghanistan | 2017  | 36296113.0 |
| ...  | ...         | ...   | ...        |
| 1411 | Zimbabwe    | 2014  | 13586707.0 |
| 1412 | Zimbabwe    | 2015  | 13814629.0 |
| 1413 | Zimbabwe    | 2016  | 14030331.0 |
| 1414 | Zimbabwe    | 2017  | 14236595.0 |
| 1415 | Zimbabwe    | 2018  | 14438802.0 |


```

1416 rows × 3 columns

```
In [642... #Taille de la base de données  
print('le tableau contient {} observation(s) ou article(s)'.format(dispo_alimentair  
print('le tableau contient {} colonne(s)'.format(dispo_alimentaire.shape[1]))
```

le tableau contient 15605 observation(s) ou article(s)

le tableau contient 18 colonne(s)

In [643...

```
display(dispo_alimentaire.head())
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)	Dispo de l gr quan per
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	1.72	
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00	
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	2.70	

In [644...

```
#Renom de colonne  
dispo_alimentaire.rename(columns = {'Zone': 'Pays bénéficiaire'})
```

Out [644...

	Pays bénéficiaire	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	1.72
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	2.70
...	...	...	...	...	...	...	...
15600	Îles Salomon	Viande de Suides	animale	NaN	NaN	45.0	4.70
15601	Îles Salomon	Viande de Volailles	animale	NaN	NaN	11.0	3.34
15602	Îles Salomon	Viande, Autre	animale	NaN	NaN	0.0	0.06
15603	Îles Salomon	Vin	vegetale	NaN	NaN	0.0	0.07
15604	Îles Salomon	Épices, Autres	vegetale	NaN	NaN	4.0	0.48

15605 rows × 18 columns

In [645...

```
print('le tableau contient {} observation(s) ou article(s)'.format(aide_alimentaire
print('le tableau contient {} colonne(s)'.format(aide_alimentaire.shape[1]))
aide_alimentaire.rename(columns = {'Pays bénéficiaire': 'Zone'})
```

le tableau contient 1475 observation(s) ou article(s)

le tableau contient 4 colonne(s)

Out [645...

	Zone	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682
1	Afghanistan	2014	Autres non-céréales	335
2	Afghanistan	2013	Blé et Farin	39224
3	Afghanistan	2014	Blé et Farin	15160
4	Afghanistan	2013	Céréales	40504
...	...	...	...	...
1470	Zimbabwe	2015	Mélanges et préparations	96
1471	Zimbabwe	2013	Non-céréales	5022
1472	Zimbabwe	2014	Non-céréales	2310
1473	Zimbabwe	2015	Non-céréales	306
1474	Zimbabwe	2013	Riz, total	64

1475 rows × 4 columns

In [646...

```
aide_alimentaire['Valeur'] = aide_alimentaire['Valeur']*1000  
display(aide_alimentaire.head())
```

	Pays bénéficiaire	Année	Produit	Valeur
0	Afghanistan	2013	Autres non-céréales	682000
1	Afghanistan	2014	Autres non-céréales	335000
2	Afghanistan	2013	Blé et Farin	39224000
3	Afghanistan	2014	Blé et Farin	15160000
4	Afghanistan	2013	Céréales	40504000

In [647...

```
print('le tableau contient {} observation(s) ou article(s)'.format(sous_nutrition.s  
print('le tableau contient {} colonne(s)'.format(sous_nutrition.shape[1]))  
display(sous_nutrition.head())
```

```
le tableau contient 1218 observation(s) ou article(s)  
le tableau contient 3 colonne(s)
```

	Zone	Année	Valeur
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5

```
In [648... #Conversion de string en valeur numérique. Le dernier argument transforme les valeurs
sous_nutrition['Valeur'] = pd.to_numeric(sous_nutrition['Valeur'], errors = 'coerce
```

```
In [649... #inplace permet de sauvegarder le changement, comme nous l'aurions fait en stockant
sous_nutrition.rename(columns = {'Valeur': 'Sous-nutrition'}, inplace = True)
```

```
In [650... display(sous_nutrition)
```

	Zone	Année	Sous-nutrition
0	Afghanistan	2012-2014	8.6
1	Afghanistan	2013-2015	8.8
2	Afghanistan	2014-2016	8.9
3	Afghanistan	2015-2017	9.7
4	Afghanistan	2016-2018	10.5
...	...	...	...
1213	Zimbabwe	2013-2015	NaN
1214	Zimbabwe	2014-2016	NaN
1215	Zimbabwe	2015-2017	NaN
1216	Zimbabwe	2016-2018	NaN
1217	Zimbabwe	2017-2019	NaN

1218 rows × 3 columns

```
In [651... #Conversion de la population, de millions à l'unité
sous_nutrition['Sous-nutrition'] = sous_nutrition['Sous-nutrition'] * 1000000
```

```
In [652... display(sous_nutrition.head())
```

	Zone	Année	Sous-nutrition
0	Afghanistan	2012-2014	8600000.0
1	Afghanistan	2013-2015	8800000.0
2	Afghanistan	2014-2016	8900000.0
3	Afghanistan	2015-2017	9700000.0
4	Afghanistan	2016-2018	10500000.0

In [653... *#Création d'une variable ne gardant les données de population à l'année 2017*

```
population_2017 = population.loc[population['Année']==2017, :]
display(population_2017)
```

	Zone	Année	Valeur
4	Afghanistan	2017	36296113.0
10	Afrique du Sud	2017	57009756.0
16	Albanie	2017	2884169.0
22	Algérie	2017	41389189.0
28	Allemagne	2017	82658409.0
...	...	...	...
1390	Venezuela (République bolivarienne du)	2017	29402484.0
1396	Viet Nam	2017	94600648.0
1402	Yémen	2017	27834819.0
1408	Zambie	2017	16853599.0
1414	Zimbabwe	2017	14236595.0

236 rows × 3 columns

In [654... *#Idem, mais pour le document sous-nutrition*

```
sous_nutrition_2017 = sous_nutrition.loc[sous_nutrition['Année']=='2016-2018', :]
display(sous_nutrition_2017)
```

	Zone	Année	Sous-nutrition
4	Afghanistan	2016-2018	10500000.0
10	Afrique du Sud	2016-2018	3100000.0
16	Albanie	2016-2018	100000.0
22	Algérie	2016-2018	1300000.0
28	Allemagne	2016-2018	NaN
...	...	...	...
1192	Venezuela (République bolivarienne du)	2016-2018	8000000.0
1198	Viet Nam	2016-2018	6500000.0
1204	Yémen	2016-2018	NaN
1210	Zambie	2016-2018	NaN
1216	Zimbabwe	2016-2018	NaN

203 rows × 3 columns

Etude de la population mondiale en état de sous-nutrition

In [655...

```
#Jointure des deux documents récemment créés, avec 'Zone' comme clé
population_sous_nutrition = pd.merge(population_2017, sous_nutrition_2017, on = 'Zo
display(population_sous_nutrition)
```


	Zone	Année_x	Valeur	Année_y	Sous-nutrition
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0
1	Afrique du Sud	2017	57009756.0	2016-2018	3100000.0
2	Albanie	2017	2884169.0	2016-2018	100000.0
3	Algérie	2017	41389189.0	2016-2018	1300000.0
4	Allemagne	2017	82658409.0	2016-2018	NaN
...	...	...	...	...	...
198	Venezuela (République bolivarienne du)	2017	29402484.0	2016-2018	8000000.0
199	Viet Nam	2017	94600648.0	2016-2018	6500000.0
200	Yémen	2017	27834819.0	2016-2018	NaN
201	Zambie	2017	16853599.0	2016-2018	NaN
202	Zimbabwe	2017	14236595.0	2016-2018	NaN

203 rows × 5 columns

In [656...

```
#Calcul, pour chaque pays, du pourcentage local de la population en sous-nutrition
population_sous_nutrition['Pourcentage sous-nutrition'] = population_sous_nutrition
display(population_sous_nutrition)
#Calcul du pourcentage mondiale de la population en situation de sous-nutrition. On
proportion_mondiale = population_sous_nutrition['Sous-nutrition'].sum()*100/populat
print(population_sous_nutrition['Valeur'].sum())
print(population_sous_nutrition['Sous-nutrition'].sum())
print(proportion_mondiale)
```

	Zone	Année_x	Valeur	Année_y	Sous-nutrition	Pourcentage sous-nutrition
0	Afghanistan	2017	36296113.0	2016-2018	10500000.0	28.928718
1	Afrique du Sud	2017	57009756.0	2016-2018	3100000.0	5.437666
2	Albanie	2017	2884169.0	2016-2018	100000.0	3.467203
3	Algérie	2017	41389189.0	2016-2018	1300000.0	3.140917
4	Allemagne	2017	82658409.0	2016-2018	NaN	NaN
...	...	...	...	...	...	...
198	Venezuela (République bolivarienne du)	2017	29402484.0	2016-2018	8000000.0	27.208586
199	Viet Nam	2017	94600648.0	2016-2018	6500000.0	6.870989
200	Yémen	2017	27834819.0	2016-2018	NaN	NaN
201	Zambie	2017	16853599.0	2016-2018	NaN	NaN
202	Zimbabwe	2017	14236595.0	2016-2018	NaN	NaN

203 rows × 6 columns

7543798779.0

535700000.0

7.1011968332354165

In [657... *#Création de la variable de la quantité journalière de calories requises par person.*
 required_kcal = 2500

In [658... *#Fusion des documents dispo_alimentaire et population, avec 'Zone' pour clé*
 data_dispo = pd.merge(dispo_alimentaire, population_2017, on = 'Zone')
 display(data_dispo)

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	1.72
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	2.70
...	...	...	...	...	...	...	...
15411	Îles Salomon	Viande de Suides	animale	NaN	NaN	45.0	4.70
15412	Îles Salomon	Viande de Volailles	animale	NaN	NaN	11.0	3.34
15413	Îles Salomon	Viande, Autre	animale	NaN	NaN	0.0	0.06
15414	Îles Salomon	Vin	vegetale	NaN	NaN	0.0	0.07
15415	Îles Salomon	Épices, Autres	vegetale	NaN	NaN	4.0	0.48

15416 rows × 20 columns

In [659... `#produit des disponibilités alimentaire par personne, par population locale afin d'`
`data_dispo['dispo_kcal(kcal/j)'] = data_dispo['Disponibilité alimentaire (Kcal/pers`

In [660... `display(data_dispo)`

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)
0	Afghanistan	Abats Comestible	animale	NaN	NaN	5.0	1.72
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	2.70
...	...	...	...	...	...	...	...
15411	Îles Salomon	Viande de Suides	animale	NaN	NaN	45.0	4.70
15412	Îles Salomon	Viande de Volailles	animale	NaN	NaN	11.0	3.34
15413	Îles Salomon	Viande, Autre	animale	NaN	NaN	0.0	0.06
15414	Îles Salomon	Vin	vegetale	NaN	NaN	0.0	0.07
15415	Îles Salomon	Épices, Autres	vegetale	NaN	NaN	4.0	0.48

15416 rows × 21 columns

```
In [661... #Obtention de la dispo alimenraire totale
somme = data_dispo['dispo_kcal(kcal/j)'].sum()
```

```
In [662... print(somme)
```

20918984627331.0

```
In [663... #Division par la quantité journalière requise, et affichage en milliards d'habitant
milliards_hab = somme/required_kcal/100000000
print(milliards_hab)
```

8.3675938509324

Etude de la population pouvant théoriquement être nourrie avec des

végétaux

```
In [664... #création d'une base de données ne gardant que les produits d'origine végétale
data_dispo_veg = data_dispo.loc[data_dispo['Origine']=='vegetale', :]
```

```
In [665... display(data_dispo_veg.head(10))
```

	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)	Dispo de ! qu: p
1	Afghanistan	Agrumes, Autres	vegetale	NaN	NaN	1.0	1.29	
2	Afghanistan	Aliments pour enfants	vegetale	NaN	NaN	1.0	0.06	
3	Afghanistan	Ananas	vegetale	NaN	NaN	0.0	0.00	
4	Afghanistan	Bananes	vegetale	NaN	NaN	4.0	2.70	
6	Afghanistan	Bière	vegetale	NaN	NaN	0.0	0.09	
7	Afghanistan	Blé	vegetale	NaN	NaN	1369.0	160.23	
8	Afghanistan	Boissons Alcooliques	vegetale	NaN	NaN	0.0	0.00	
9	Afghanistan	Café	vegetale	NaN	NaN	0.0	0.00	
10	Afghanistan	Coco (Incl Coprah)	vegetale	NaN	NaN	0.0	0.00	
12	Afghanistan	Céréales, Autres	vegetale	NaN	NaN	0.0	0.00	

10 rows × 21 columns

```
In [666... #Même calculs que précédemment, mais sur la nouvelle base de données
veg_dispo = data_dispo_veg['dispo_kcal(kcal/j)'].sum()
print(veg_dispo)
#Nombre d'habitants pouvant théoriquement être nourris avec des aliments d'origine
sum_hab_veg_milliards = veg_dispo/required_kcal/1000000000
print(sum_hab_veg_milliards)
```

17260764211501.0

6.9043056846004

```
In [667... dispo_totale = dispo_alimentaire['Disponibilité intérieure'].sum()
print(dispo_totale)
```

9848994.0

Part d'utilisation de la nourriture dans le monde

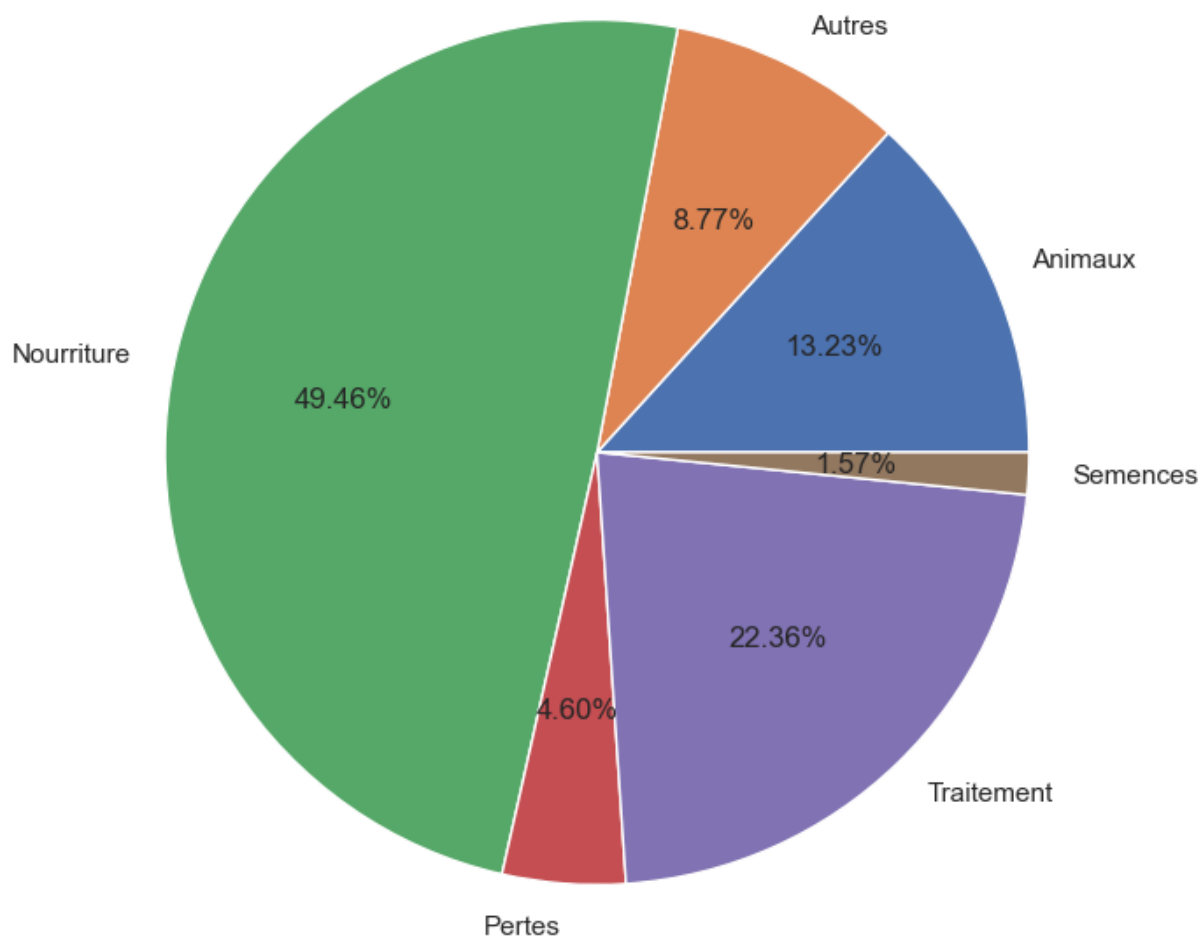
```
In [668... #Création de variables totalisant les utilisations respectives des utilisations de
Animaux = dispo_alimentaire['Aliments pour animaux'].sum()
Autres = dispo_alimentaire['Autres Utilisations'].sum()
Nourriture = dispo_alimentaire['Nourriture'].sum()
Pertes = dispo_alimentaire['Pertes'].sum()
Semences = dispo_alimentaire['Semences'].sum()
Traitement = dispo_alimentaire['Traitement'].sum()
```

```
In [669... #Création d'un tableau via Numpy pour faciliter Le traçage d'un graphique
import numpy as np
import matplotlib.pyplot as plt
totaux_int = np.array([[ 'Animaux', Animaux], [ 'Autres', Autres], [ 'Nourriture', Nou
print(totaux_int)
```

```
[[ 'Animaux' '1304245.0']
[ 'Autres' '865023.0']
[ 'Nourriture' '4876258.0']
[ 'Pertes' '453698.0']
[ 'Traitement' '2204687.0']
[ 'Semences' '154681.0']]
```

```
In [708... #Répartition de la disponibilité intérieure mondiale
sns.set_theme(rc={'figure.figsize':(12,8)})
plt.pie(x=totaux_int[:, 1], labels=totaux_int[:, 0], autopct='%.2f%%')
plt.title('Répartition de la disponibilité intérieure mondiale')
plt.show()
```

Répartition de la disponibilité intérieure mondiale



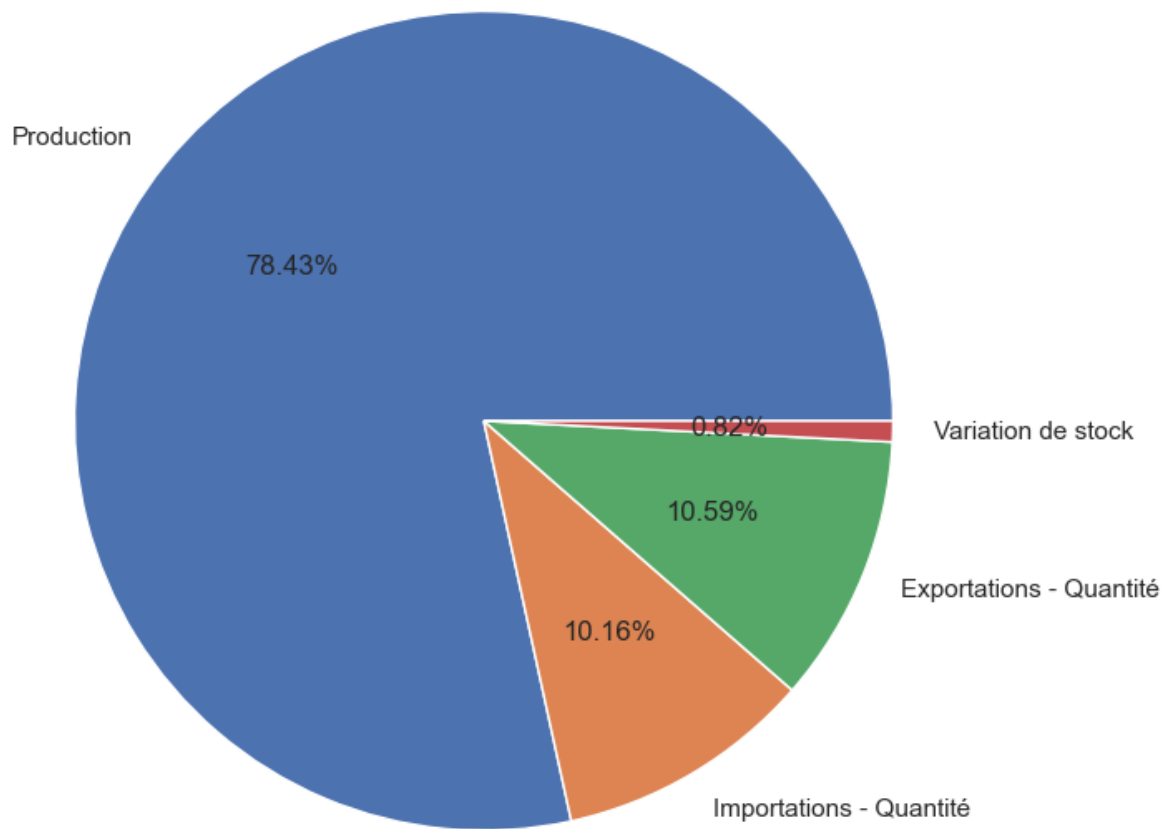
```
In [707... #Création de variables totalisant les utilisations respectives des utilisations de
Production = dispo_alimentaire['Production'].sum()
Imports = dispo_alimentaire['Importations - Quantité'].sum()
Exports = dispo_alimentaire['Exportations - Quantité'].sum()
Variations = dispo_alimentaire['Variation de stock'].sum()

Transferts = np.array([[ 'Production', Production], [ 'Importations - Quantité', Imports], [ 'Exportations - Quantité', Exports], [ 'Variation de stock', Variations]])
print(Transferts)

sns.set_theme(rc={'figure.figsize':(8,8)})
plt.pie(x=Transferts[:, 1], labels=Transferts[:, 0], autopct='%.2f%%')
plt.title('Répartition de la disponibilité intérieure mondiale')
plt.show()

[[ 'Production' '10009680.0']
[ 'Importations - Quantité' '1296053.0']
[ 'Exportations - Quantité' '1352158.0']
[ 'Variation de stock' '104402.0']]
```

Répartition de la disponibilité intérieure mondiale



Etude de l'utilisation des céréales dans le monde

```
In [671... #Création d'une liste contenant l'ensemble des types de céréales mentionnées dans L
liste_cereales = ['Blé', 'Riz (Eq Blanchi)', 'Orge', 'Maïs', 'Seigle', 'Avoine',
                  'Millet', 'Sorgho', 'Céréales, Autres']
```

```
In [672... #Création d'une base de données ne gardant que les produits de type céréale
df_tri = []
for i in liste_cereales:
    df_tri.append(dispo_alimentaire.loc[dispo_alimentaire['Produit']==i,:])
data_cereales = pd.concat(df_tri)
display(data_cereales)
```


	Zone	Produit	Origine	Aliments pour animaux	Autres Utilisations	Disponibilité alimentaire (Kcal/ personne/ jour)	Disponibilité alimentaire en quantité (kg/ personne/ an)	Dis c qu
7	Afghanistan	Blé	vegetale	NaN	NaN	1369.0	160.23	
72	Afrique du Sud	Blé	vegetale	37.0	NaN	492.0	60.13	
167	Albanie	Blé	vegetale	18.0	130.0	1056.0	138.64	
259	Algérie	Blé	vegetale	545.0	820.0	1424.0	185.42	
352	Allemagne	Blé	vegetale	7494.0	774.0	654.0	83.41	
...	...	...	...	...	...	...	...	
15173	Émirats arabes unis	Céréales, Autres	vegetale	1.0	1.0	0.0	0.00	
15266	Équateur	Céréales, Autres	vegetale	1.0	NaN	0.0	0.05	
15361	États-Unis d'Amérique	Céréales, Autres	vegetale	77.0	NaN	5.0	0.62	
15454	Éthiopie	Céréales, Autres	vegetale	0.0	469.0	254.0	26.52	
15545	Îles Salomon	Céréales, Autres	vegetale	NaN	NaN	0.0	0.00	

1497 rows × 18 columns

```
In [673... #Mise à jour de la base de données, ne conservant que les utilisation céréalières p
part_cereales = data_cereales.groupby('Produit')[['Nourriture', 'Aliments pour anim
part_cereales = part_cereales.reset_index()
```

```
In [674... #Création des valeurs de quantité et pourcentage pour les deux catégories
part_cereales['Total'] = part_cereales['Nourriture'] + part_cereales['Aliments pour
part_cereales['Part Alimentation humaine (%)'] = round(part_cereales['Nourriture']*
part_cereales['Part Alimentation pour animaux (%)'] = round(part_cereales['Aliments
```

```
In [675... display(part_cereales)
```

	Produit	Nourriture	Aliments pour animaux	Total	Part Alimentation humaine (%)	Part Alimentation pour animaux (%)
0	Avoine	3903.0	16251.0	20154.0	19.37	80.63
1	Blé	457824.0	129668.0	587492.0	77.93	22.07
2	Céréales, Autres	5324.0	19035.0	24359.0	21.86	78.14
3	Maïs	125184.0	546116.0	671300.0	18.65	81.35
4	Millet	23040.0	3306.0	26346.0	87.45	12.55
5	Orge	6794.0	92658.0	99452.0	6.83	93.17
6	Riz (Eq Blanchi)	377286.0	33594.0	410880.0	91.82	8.18
7	Seigle	5502.0	8099.0	13601.0	40.45	59.55
8	Sorgho	24153.0	24808.0	48961.0	49.33	50.67

In [676...

```
#Formatage de la base de donnée pour faciliter la création d'un graphique
part_cereales = part_cereales.melt(id_vars = 'Produit', value_vars = ['Nourriture',
display(part_cereales)
```

	Produit	variable	value
0	Avoine	Nourriture	3903.0
1	Blé	Nourriture	457824.0
2	Céréales, Autres	Nourriture	5324.0
3	Maïs	Nourriture	125184.0
4	Millet	Nourriture	23040.0
5	Orge	Nourriture	6794.0
6	Riz (Eq Blanchi)	Nourriture	377286.0
7	Seigle	Nourriture	5502.0
8	Sorgho	Nourriture	24153.0
9	Avoine	Aliments pour animaux	16251.0
10	Blé	Aliments pour animaux	129668.0
11	Céréales, Autres	Aliments pour animaux	19035.0
12	Maïs	Aliments pour animaux	546116.0
13	Millet	Aliments pour animaux	3306.0
14	Orge	Aliments pour animaux	92658.0
15	Riz (Eq Blanchi)	Aliments pour animaux	33594.0
16	Seigle	Aliments pour animaux	8099.0
17	Sorgho	Aliments pour animaux	24808.0

In [677... *#Création du graphique d'utilisation des céréales*

```

import seaborn as sns
#On retire Le style Seaborn défini précédemment pour que celui-ci n'interfère pas a
sns.reset_defaults()
sns.set_theme(rc={'figure.figsize':(12,8)})
ax = sns.barplot(data=part_cereales, x='Produit', y='value', ci=None, hue='variable')
for cont in ax.containers:
    ax.bar_label(cont, fontsize = 8)
plt.legend(title='Utilisation')
plt.ylabel('Quantité en millier de tonnes', fontsize = 18)
plt.xlabel('Type de céréale', fontsize = 18)
plt.show()

```

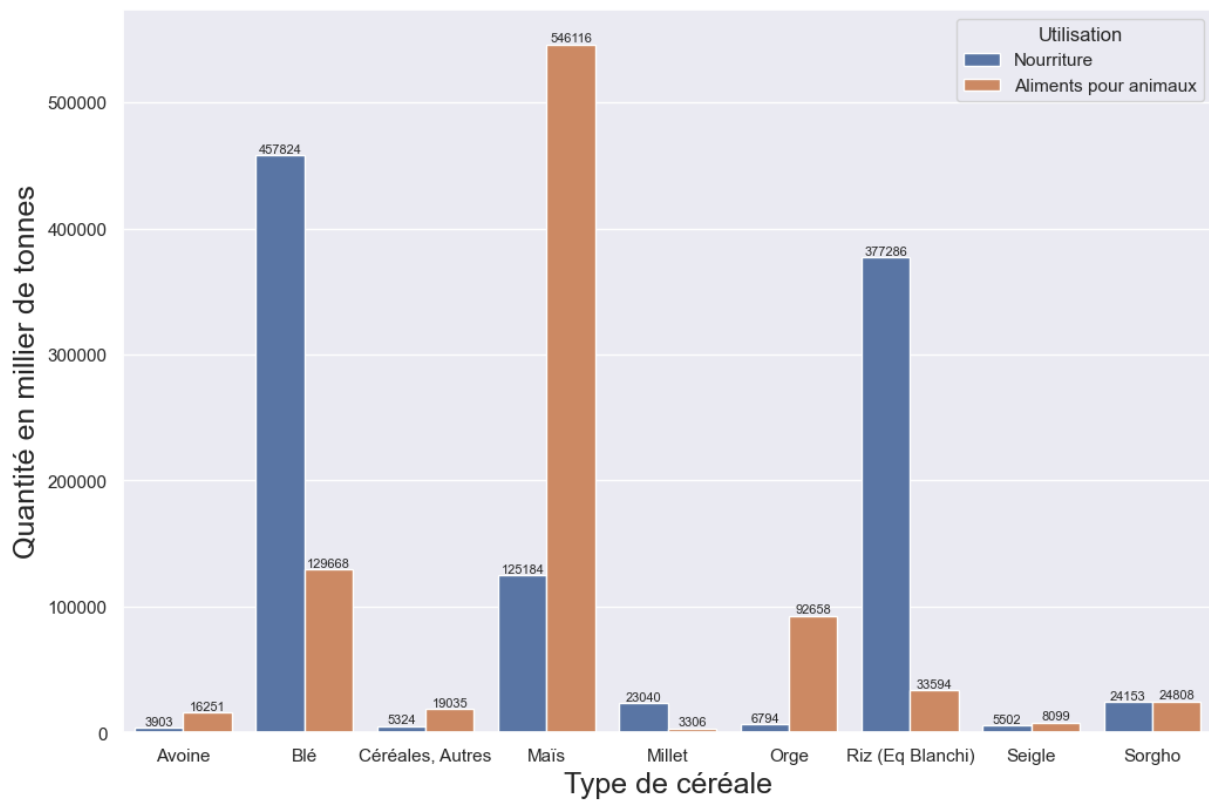
C:\Users\agall\AppData\Local\Temp\ipykernel_33244\2589117029.py:6: FutureWarning:

The `ci` parameter is deprecated. Use `errorbar=None` for the same effect.

```

ax = sns.barplot(data=part_cereales, x='Produit', y='value', ci=None, hue='variable')

```



Sélection des 10 pays étant le plus en état de sous-nutrition

In [678...

```
#Tri décroissant de la population en état de sous-nutrition par pays
tri = population_sous_nutrition.sort_values(['Pourcentage sous-nutrition'], ascendi
#10 premières valeurs
tri2 = tri.head(10)
tri2 = tri2.reset_index()
#Changement de nom de République Démocratique de Corée, pour que le nom ne se super
tri2.loc[1, 'Zone'] = 'DRPK'
display(tri2)
```

	index	Zone	Année_x	Valeur	Année_y	Sous-nutrition	Pourcentage sous-nutrition
0	78	Haïti	2017	10982366.0	2016-2018	5300000.0	48.259182
1	157	DRPK	2017	25429825.0	2016-2018	12000000.0	47.188685
2	108	Madagascar	2017	25570512.0	2016-2018	10500000.0	41.062924
3	103	Libéria	2017	4702226.0	2016-2018	1800000.0	38.279742
4	100	Lesotho	2017	2091534.0	2016-2018	800000.0	38.249438
5	183	Tchad	2017	15016753.0	2016-2018	5700000.0	37.957606
6	161	Rwanda	2017	11980961.0	2016-2018	4200000.0	35.055619
7	121	Mozambique	2017	28649018.0	2016-2018	9400000.0	32.810898
8	186	Timor-Leste	2017	1243258.0	2016-2018	400000.0	32.173531
9	0	Afghanistan	2017	36296113.0	2016-2018	10500000.0	28.928718

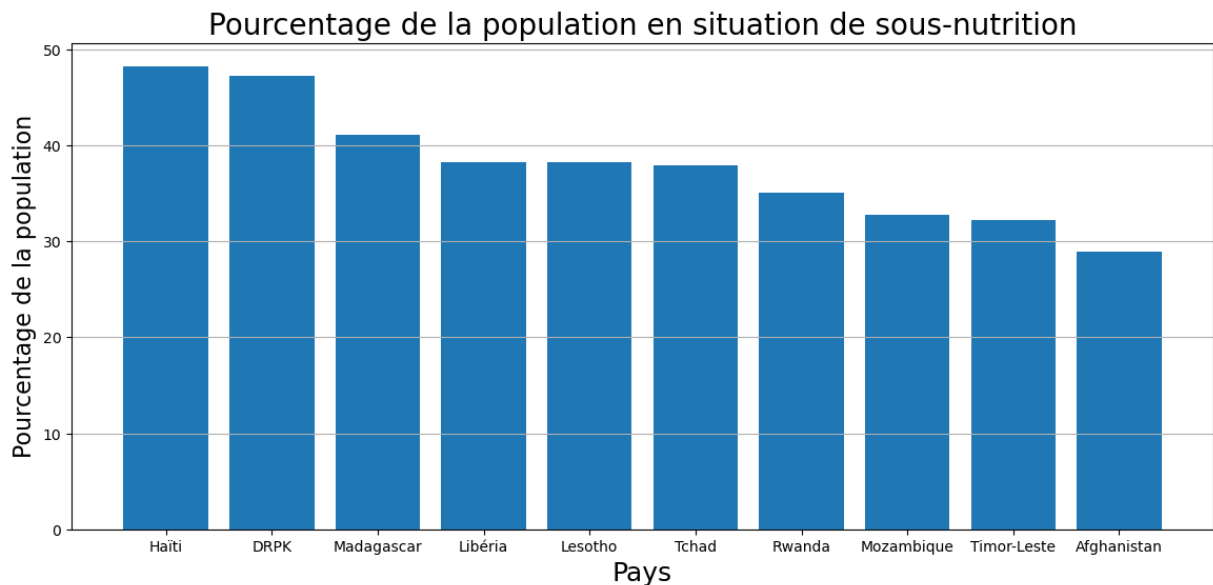
In [679...

```

#Création du graphique correspondant
#On retire le style Seaborn défini précédemment pour que celui-ci n'interfère pas a
sns.reset_defaults()

plt.figure(figsize = (14, 6))
plt.bar(height = tri2['Pourcentage sous-nutrition'], x = tri2['Zone'])
plt.title('Pourcentage de la population en situation de sous-nutrition', fontsize =
plt.xlabel('Pays', fontsize = 18)
plt.ylabel('Pourcentage de la population', fontsize = 16)
plt.grid(axis = 'y')
plt.show()

```



Etude des 10 pays ayant le plus bénéficié de

l'aide alimentaire

In [680...

```
#Somme des valeurs de L'aide alimentaire reçue entre 2013 et 2016
aide_2013_2016 = aide_alimentaire.groupby('Pays bénéficiaire').agg({'Valeur': 'sum'})
display(aide_2013_2016)
```

	Valeur
Pays bénéficiaire	
Afghanistan	185452000
Algérie	81114000
Angola	5014000
Bangladesh	348188000
Bhoutan	2666000
...	...
Zambie	3026000
Zimbabwe	62570000
Égypte	1122000
Équateur	1362000
Éthiopie	1381294000

76 rows × 1 columns

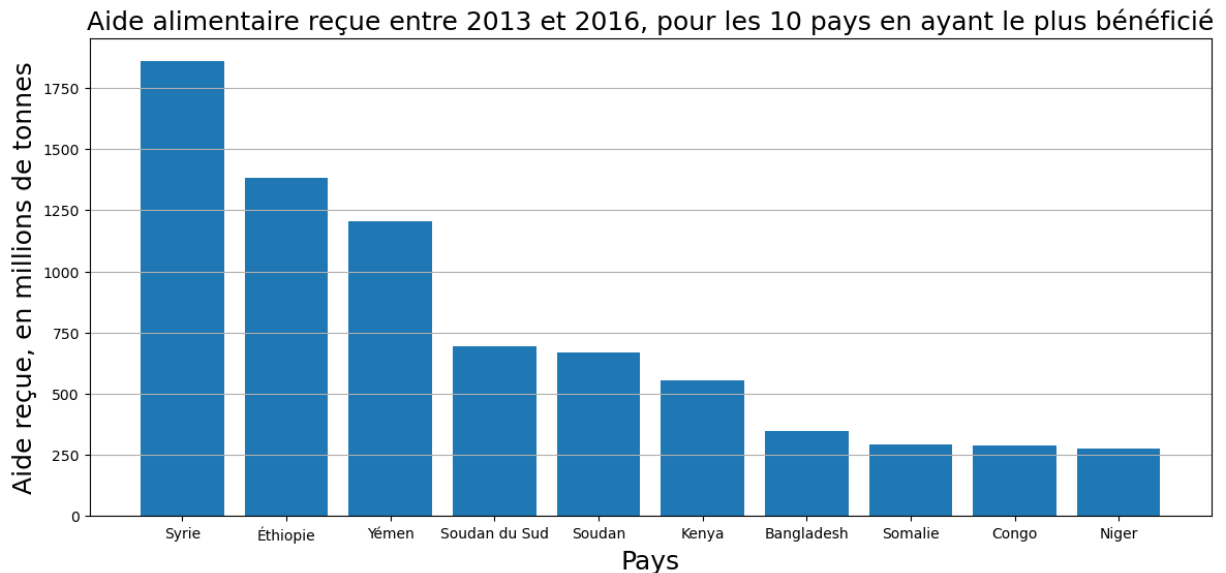
In [681...

```
#On ne garde que les 10 premières lignes
tri_final = aide_2013_2016.sort_values('Valeur', ascending = False).head(10)
tri_final = tri_final.reset_index()

#Renom des pays au nom trop long, pour éviter la juxtaposition
tri_final.loc[0, 'Pays bénéficiaire'] = 'Syrie'
tri_final.loc[8, 'Pays bénéficiaire'] = 'Congo'
display(tri_final)

#Traçage du graphique
plt.figure(figsize = (14, 6))
plt.bar(height = tri_final['Valeur']/1000000, x = tri_final['Pays bénéficiaire'])
plt.title('Aide alimentaire reçue entre 2013 et 2016, pour les 10 pays en ayant le')
plt.xlabel('Pays', fontsize = 18)
plt.ylabel('Aide reçue, en millions de tonnes', fontsize = 18)
plt.grid(axis = 'y')
plt.show()
```

	Pays bénéficiaire	Valeur
0	Syrie	1858943000
1	Éthiopie	1381294000
2	Yémen	1206484000
3	Soudan du Sud	695248000
4	Soudan	669784000
5	Kenya	552836000
6	Bangladesh	348188000
7	Somalie	292678000
8	Congo	288502000
9	Niger	276344000



Evolution des 5 pays ayant le plus réceptionné d'aide alimentaire entre 2013 et 2016

```
In [682... df_aide_pays_annee = aide_alimentaire.groupby(['Pays bénéficiaire', 'Année']).agg({'  
df_aide_pays_annee = df_aide_pays_annee.reset_index()  
display(df_aide_pays_annee )
```

	Pays bénéficiaire	Année	Valeur
0	Afghanistan	2013	128238000
1	Afghanistan	2014	57214000
2	Algérie	2013	35234000
3	Algérie	2014	18980000
4	Algérie	2015	17424000
...	...	...	...
223	Égypte	2013	1122000
224	Équateur	2013	1362000
225	Éthiopie	2013	591404000
226	Éthiopie	2014	586624000
227	Éthiopie	2015	203266000

228 rows × 3 columns

In [683...

```
#Sélection des 5 pays ayant le plus bénéficié de l'aide alimentaire
tri_final = aide_2013_2016.sort_values('Valeur', ascending = False).head(10)
tri_final = tri_final.reset_index()
top_5 = tri_final.head(5)
#Fusion de base de données, afin d'inclure les années de réception de l'aide alimen
top_5_annee = pd.merge(top_5, df_aide_pays_annee, on = 'Pays bénéficiaire')
display(top_5_annee)
```


	Pays bénéficiaire	Valeur_x	Année	Valeur_y
0	République arabe syrienne	1858943000	2013	563566000
1	République arabe syrienne	1858943000	2014	651870000
2	République arabe syrienne	1858943000	2015	524949000
3	République arabe syrienne	1858943000	2016	118558000
4	Éthiopie	1381294000	2013	591404000
5	Éthiopie	1381294000	2014	586624000
6	Éthiopie	1381294000	2015	203266000
7	Yémen	1206484000	2013	264764000
8	Yémen	1206484000	2014	103840000
9	Yémen	1206484000	2015	372306000
10	Yémen	1206484000	2016	465574000
11	Soudan du Sud	695248000	2013	196330000
12	Soudan du Sud	695248000	2014	450610000
13	Soudan du Sud	695248000	2015	48308000
14	Soudan	669784000	2013	330230000
15	Soudan	669784000	2014	321904000
16	Soudan	669784000	2015	17650000

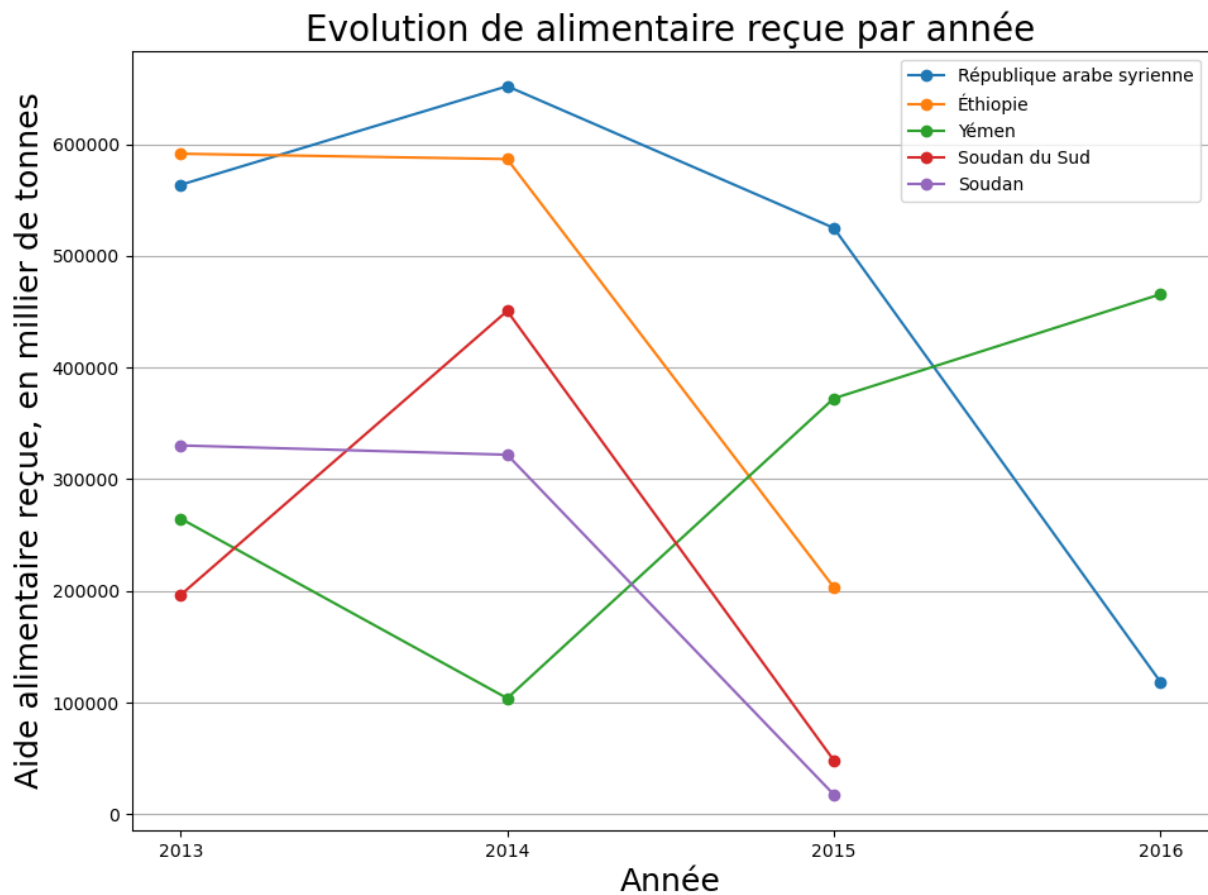
In [684...

#Traçage du graphique correspondant

```

plt.figure(figsize = (11, 8))
for pays in top_5_annee['Pays bénéficiaire'].unique():
    df = top_5_annee.loc[top_5_annee['Pays bénéficiaire']==pays, :]
    plt.plot(df['Année'], df['Valeur_y']/1000, '-o', label = pays)
plt.legend()
plt.title('Evolution de alimentaire reçue par année', fontsize = 20)
plt.xticks([2013, 2014, 2015, 2016])
plt.xlabel('Année', fontsize = 18)
plt.ylabel('Aide alimentaire reçue, en millier de tonnes', fontsize = 18)
plt.grid(axis = 'y')
plt.show()

```



10 pays ayant la plus faible disposition alimentaire

In [685...

```
#Calcul des pays ayant le plus faible disposition alimentaire
```

```
tri_disp_alimentaire = dispo_alimentaire.groupby('Zone').agg({'Disponibilité alimen
tri_disp_alimentaire = tri_disp_alimentaire.sort_values('Disponibilité alimentaire
tri_disp_alimentaire = tri_disp_alimentaire.reset_index()
```

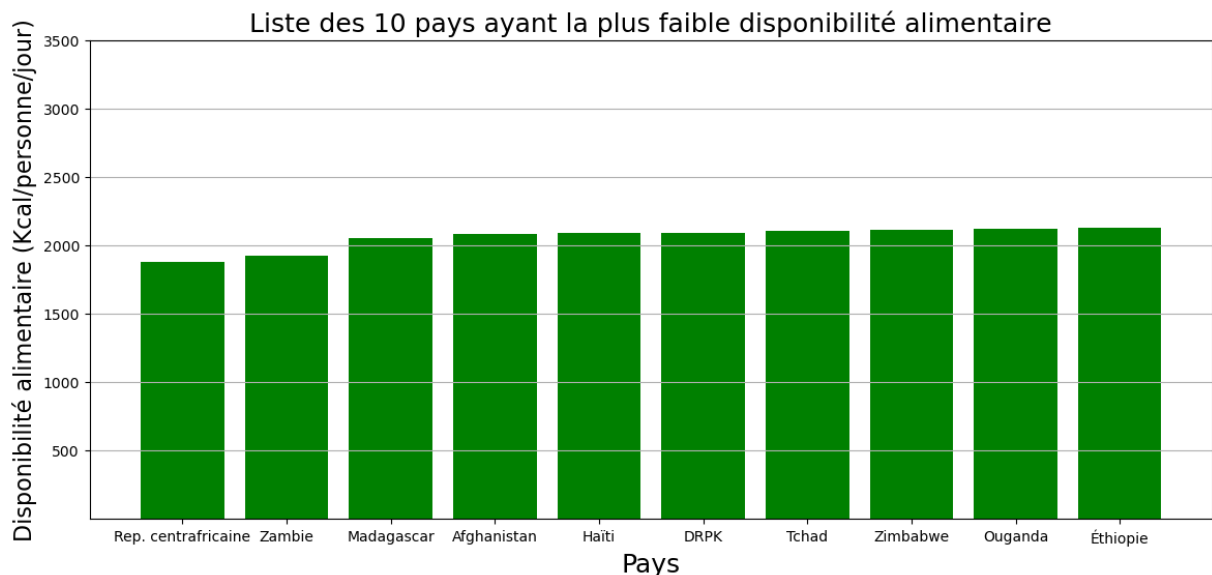
```
#Renom des pays ayant un nom trop Long
```

```
tri_disp_alimentaire.loc[0, 'Zone'] = 'Rep. centrafricaine'
tri_disp_alimentaire.loc[5, 'Zone'] = 'DRPK'
display(tri_disp_alimentaire)
```

	Zone	Disponibilité alimentaire (Kcal/personne/jour)
0	Rep. centrafricaine	1879.0
1	Zambie	1924.0
2	Madagascar	2056.0
3	Afghanistan	2087.0
4	Haïti	2089.0
5	DRPK	2093.0
6	Tchad	2109.0
7	Zimbabwe	2113.0
8	Ouganda	2126.0
9	Éthiopie	2129.0

In [686...

```
#Traçage du graphique
plt.figure(figsize = (14, 6))
plt.bar(height = tri_disp_alimentaire['Disponibilité alimentaire (Kcal/personne/jou
plt.title('Liste des 10 pays ayant la plus faible disponibilité alimentaire', fonts
plt.xlabel('Pays', fontsize = 18)
plt.ylabel('Disponibilité alimentaire (Kcal/personne/jour)', fontsize = 16)
plt.yticks([500, 1000, 1500, 2000, 2500, 3000, 3500])
plt.grid(axis = 'y')
plt.show()
```



10 pays ayant la plus forte disposition alimentaire

In [687...

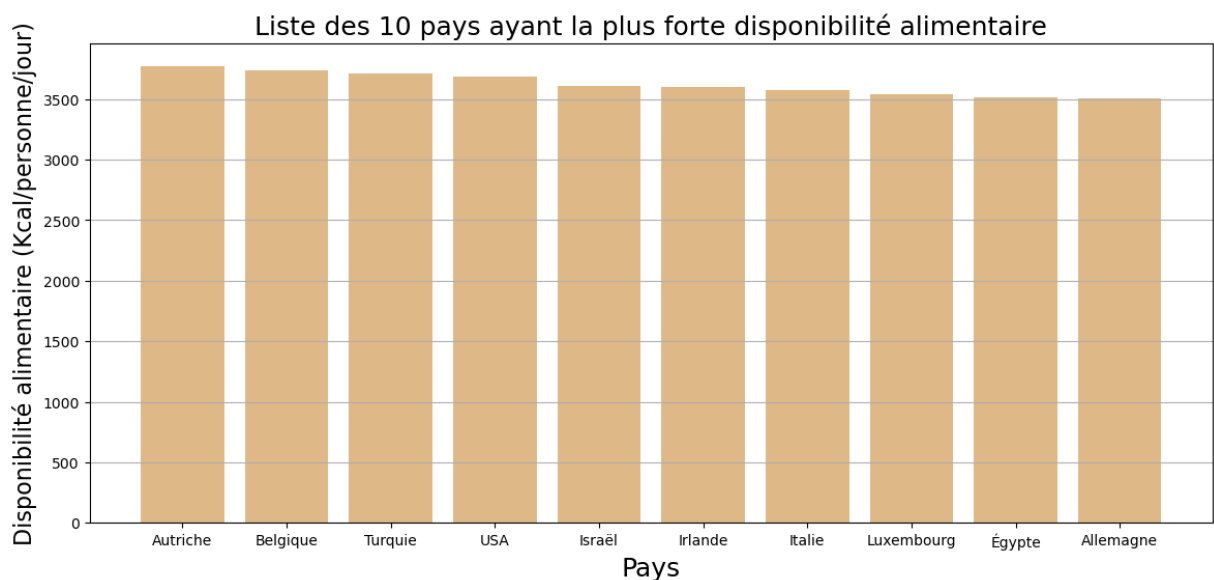
```
#Même calcul que précédemment, mais avec un tri décroissant
```

```
tri_disp_alimentaire_2 = dispo_alimentaire.groupby('Zone').agg({'Disponibilité alim  
top10 = tri_disp_alimentaire_2.sort_values('Disponibilité alimentaire (Kcal/personn  
top10 = top10.reset_index()  
top10.loc[3, "Zone"] = 'USA'  
display(top10)
```

	Zone	Disponibilité alimentaire (Kcal/personne/jour)
0	Autriche	3770.0
1	Belgique	3737.0
2	Turquie	3708.0
3	USA	3682.0
4	Israël	3610.0
5	Irlande	3602.0
6	Italie	3578.0
7	Luxembourg	3540.0
8	Égypte	3518.0
9	Allemagne	3503.0

In [688...

```
#Traçage du graphique  
plt.figure(figsize = (14, 6))  
plt.bar(height = top10['Disponibilité alimentaire (Kcal/personne/jour)'], x = top10  
plt.title('Liste des 10 pays ayant la plus forte disponibilité alimentaire', fontsi  
plt.xlabel('Pays', fontsize = 18)  
plt.ylabel('Disponibilité alimentaire (Kcal/personne/jour)', fontsize = 16)  
plt.grid(axis = 'y')  
plt.show()
```



Etude sur le manioc en Thaïlande

In [689...

```
#Etude de La sous-nutrition en Thaïlande, année après année

ss_nut_thai = sous_nutrition.loc[sous_nutrition['Zone']=='Thaïlande', :]
ss_nut_thai = ss_nut_thai.reset_index()
pop_thai = population.loc[population['Zone']=='Thaïlande', :]
ss_nut_thai.loc[0, 'Année'] = 2013
ss_nut_thai.loc[1, 'Année'] = 2014
ss_nut_thai.loc[2, 'Année'] = 2015
ss_nut_thai.loc[3, 'Année'] = 2016
ss_nut_thai.loc[4, 'Année'] = 2017
ss_nut_thai.loc[5, 'Année'] = 2018
display(ss_nut_thai)
```

	index	Zone	Année	Sous-nutrition
0	1110	Thaïlande	2013	6200000.0
1	1111	Thaïlande	2014	6000000.0
2	1112	Thaïlande	2015	5900000.0
3	1113	Thaïlande	2016	6000000.0
4	1114	Thaïlande	2017	6200000.0
5	1115	Thaïlande	2018	6500000.0

In [690...

```
#Fusion de La base de donnée précédente avec celle de La population année par année
df_thai = pd.merge(ss_nut_thai, pop_thai, on = ['Année', 'Zone'], how = 'left')
```

In [691...

```
df_thai = df_thai.rename(columns={'Valeur':'Population totale'})
```

In [692...

```
#Calcul du pourcentage de La population en sous-alimentation en Thaïlande
df_thai['Pourcentage population malnutrition'] = df_thai['Sous-nutrition']*100/df_t
```

In [693... `display(df_thai)`

	index	Zone	Année	Sous-nutrition	Population totale	Pourcentage population malnutrition
0	1110	Thaïlande	2013	6200000.0	68144518.0	9.098311
1	1111	Thaïlande	2014	6000000.0	68438746.0	8.766964
2	1112	Thaïlande	2015	5900000.0	68714511.0	8.586250
3	1113	Thaïlande	2016	6000000.0	68971308.0	8.699270
4	1114	Thaïlande	2017	6200000.0	69209810.0	8.958268
5	1115	Thaïlande	2018	6500000.0	69428453.0	9.362156

In [694... `#Création d'une base de donnée pour la disponibilité alimentaire en Thaïlande, Limi`
`import_export_thai = dispo_alimentaire.loc([(dispo_alimentaire['Produit']=='Manioc')]`

In [695... `display(import_export_thai )`

	Exportations - Quantité	Disponibilité intérieure
13809	25214.0	6264.0

In [696... `#Création d'un graphique représentant la distribution de l'utilisation locale du ma`
`display(import_export_thai )`
`values = import_export_thai.values.flatten()`
`plt.pie(x=values, autopct='%.2f%%')`

	Exportations - Quantité	Disponibilité intérieure
13809	25214.0	6264.0

Out[696... (`<matplotlib.patches.Wedge at 0x18540097b10>`,
`<matplotlib.patches.Wedge at 0x18540097ed0>`],
`[Text(-0.8919534431532444, 0.6437538778501238, '')]`,
`Text(0.8919536529148355, -0.6437535872147676, '')]`,
`[Text(-0.4865200599017696, 0.35113847882734023, '80.10%')]`,
`Text(0.486520174317183, -0.3511383202989641, '19.90%')]`)

In [697... `dispo_alim_thailand = dispo_alimentaire.loc[dispo_alimentaire['Zone']=='Thaïlande',`

In [698... `print(dispo_alim_thailand['Disponibilité alimentaire (Kcal/personne/jour)'].sum())`

2785.0

Complément d'étude : accès aux protéines dans le monde

In [719...

#Etude disponibilité protéines dans Le monde

```
tri_prot = dispo_alimentaire.groupby('Zone').agg({'Disponibilité de protéines en qu
tri_prot = tri_prot.sort_values('Disponibilité de protéines en quantité (g/personne
prot_top5 = tri_prot.head(5)
prot_top5 = prot_top5.reset_index()
display(prot_top5)
prot_flop5 = tri_prot.tail(5)
prot_flop5 = prot_flop5.reset_index()
display(prot_flop5)
list_concat = [prot_top5, prot_flop5]
total_prot = pd.concat(list_concat)
total_prot = total_prot.reset_index()
display(total_prot)
```

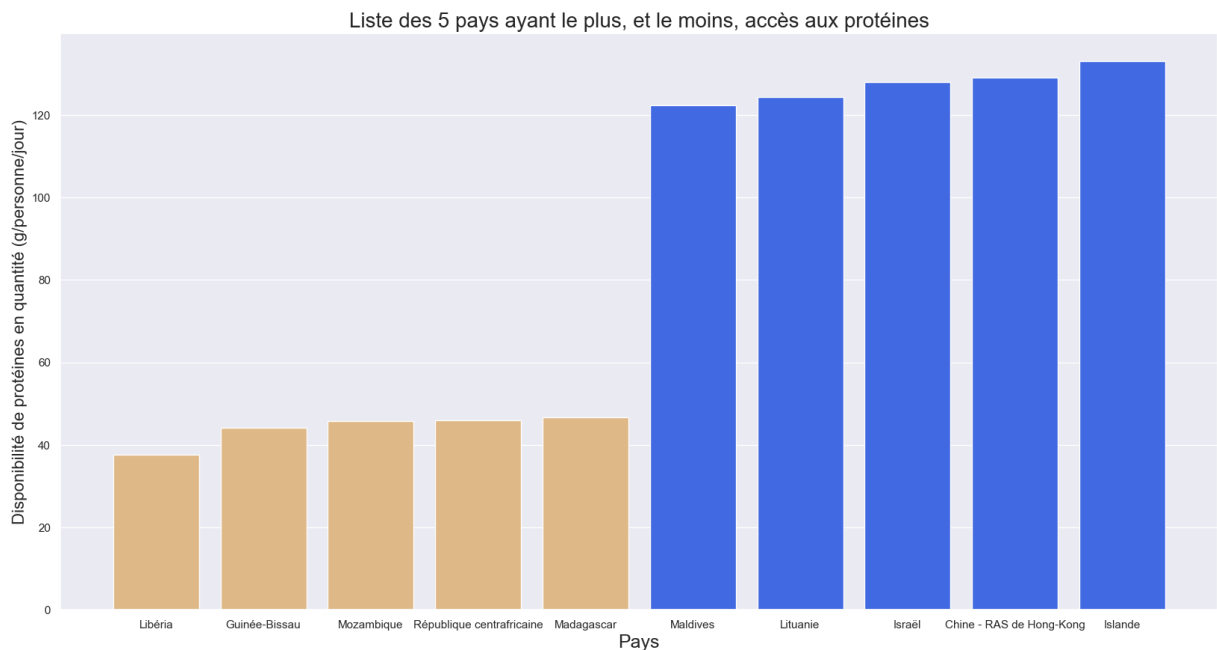
	Zone	Disponibilité de protéines en quantité (g/personne/jour)
0	Libéria	37.66
1	Guinée-Bissau	44.05
2	Mozambique	45.68
3	République centrafricaine	46.04
4	Madagascar	46.69

	Zone	Disponibilité de protéines en quantité (g/personne/jour)
0	Maldives	122.32
1	Lituanie	124.36
2	Israël	128.00
3	Chine - RAS de Hong-Kong	129.07
4	Islande	133.06

	index	Zone	Disponibilité de protéines en quantité (g/personne/jour)
0	0	Libéria	37.66
1	1	Guinée-Bissau	44.05
2	2	Mozambique	45.68
3	3	République centrafricaine	46.04
4	4	Madagascar	46.69
5	0	Maldives	122.32
6	1	Lituanie	124.36
7	2	Israël	128.00
8	3	Chine - RAS de Hong-Kong	129.07
9	4	Islande	133.06

In [731...

```
plt.figure(figsize = (20, 10))
color_bar = ['burlywood', 'burlywood', 'burlywood', 'burlywood', 'burlywood', 'royalblue', 'royalblue', 'royalblue', 'royalblue', 'royalblue']
plt.bar(height = total_prot['Disponibilité de protéines en quantité (g/personne/jour)'], color = color_bar)
plt.title("Liste des 5 pays ayant le plus, et le moins, accès aux protéines", fontsize = 14)
plt.xlabel('Pays', fontsize = 18)
plt.ylabel('Disponibilité de protéines en quantité (g/personne/jour)', fontsize = 14)
plt.grid(axis = 'x')
plt.show()
```



In []: